

CodioProjectFileEditFindViewToolsEducationHelpConfigure...Project Index (static)Configure...

sync.pyTerminalx

Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 5.19.0-1028-aws x86_64)

* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage

* Canonical Livepatch is available for installation.
- Reduce system reboots and improve kernel security. Activate at: https://ubuntu.com/livepatch

*
* Welcome to the Codio Terminal!
*
* https://docs.codio.com/develop/develop/ide/boxes/overview
*
* Your Codio Box domain is: analogorlando-edgarbuenos.codio.io
*
Last login: Fri Jul 21 20:00:11 2023 from 192.168.11.179
codio@analogorlando-edgarbuenos:~/workspace\$ python3 sync.py 2
codio@analogorlando-edgarbuenos:~/workspace\$

Guide x

Collapse

Async vs Sync

In a typical **synchronous application**, each task must complete before the next one can start. This can be inefficient when dealing with I/O operations like network requests, where the program spends a lot of time waiting for responses.

Asynchronous programming allows you to perform other tasks while waiting for I/O operations to complete, resulting in more efficient use of resources.

Python's `asyncio` library allows you to write single-threaded concurrent code using coroutines, multiplexing I/O access over sockets and other resources, running network clients and servers, and other related primitives.

Asyncynchronous

While using DALL-E's API, you might need to generate multiple images at once. If you do it synchronously, your application will have to wait for each API call to complete before making the next one. This is inefficient, especially if the API calls take a long time to process.

By making asynchronous API calls, you can send multiple requests to the API at once without having to wait for each one to complete. This can significantly speed up your application if you need to make a lot of API calls.

Synchronous example

In order to compare the time between both calls, we are going to time our previous chain request. Copy and paste the code in this page in the `sync.py` file on the left. To control the time we have to use the following code. The code will start a timer then end it when we are done testing and print the result. here is a sample code, just for your visual learning.

```
import time

start_time = time.time()

# Your existing code here

end_time = time.time()
execution_time = end_time - start_time
print(f"Execution time: {execution_time} seconds")
```

By using `time.time()` function, we capture the start time before the execution of your code and the end time after the execution. Then, we calculate the difference between the two timestamps to get the execution time in seconds. Finally, we print the execution time.

To keep it consistent let's basically use the same Python code we have been using in this assignment.

```
import os
import openai
import requests
import secret
import time

openai.api_key = secret.api_key

# Set the prompts
prompts = ["robot dog in a lab", "robot dog exploring the city", "robot dog watching"]

start_time = time.time() # Start measuring execution time

# Generate and save the images
for i, prompt in enumerate(prompts):
    try:
        response = openai.Image.create(
            prompt=prompt,
            n=1,
            size="256x256"
        )

        # Get the image URL from the response
        image_url = response['data'][0]['url']

        # Download and save the image
        img_data = requests.get(image_url).content
        with open(f"robot_dog_journey_{i+1}.jpg", 'wb') as handler:
            handler.write(img_data)

    except requests.exceptions.RequestException as e:
        # This will catch any general network error
        print(f"Network error: {e}")

    except openai.api_errors.APIError as e:
        # This will catch any error returned by the OpenAI API
        print(f"API error: {e}")

    except Exception as e:
        # This is a catch-all for any other exceptions
        print(f"Unexpected error: {e}")

end_time = time.time() # End measuring execution time
execution_time = end_time - start_time
print(f"Execution time: {execution_time} seconds")
```

TRY IT SYNC!

In Python, what library allows you to write single-threaded concurrent code using coroutines, multiplexing I/O access over sockets and other resources, running network clients and servers, and other related primitives?

☐ multiprocessing

☐ multithreading

☒ asyncio

☐ concurrent.futures

Python's `asyncio` library is designed to write single-threaded concurrent code using coroutines, multiplexing I/O access over sockets and other resources. It's used for asynchronous I/O and allows you to structure your code in a more straightforward way, while also making more efficient use of system resources.

Check !!!

Next

1/1